Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

IIC2133 – Estructuras de Datos y Algoritmos

2026 - 1

Taller 2

Objetivos

- Implementar algoritmos de ordenamiento en distintos escenarios.
- Analizar la complejidad temporal segun el tamaño de los datos.
- Modificar algoritmos de sorting para resolver problemas especificos.

Introducción

Después de varias semanas ayudando al Profesor Oak a organizar información sobre los Pokémon, ha surgido un nuevo desafío en el laboratorio. Investigadores de distintas regiones han comenzado a enviar grandes cantidades de datos sobre Pokémon, incluyendo atributos como nivel, velocidad y ataque, entre muchos otros.

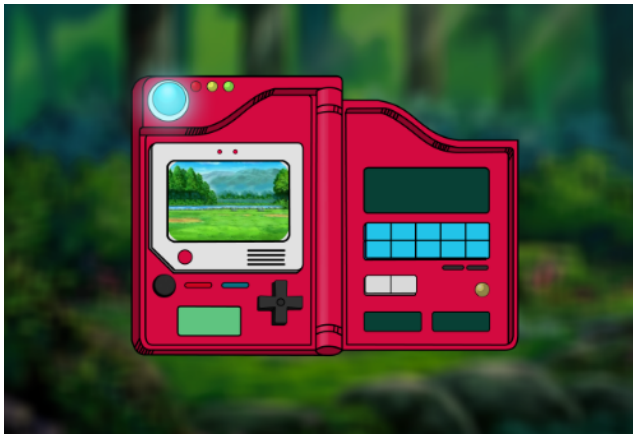
El problema es que toda esta información llega desordenada, y organizarla manualmente podría tomar demasiado tiempo. Para poder analizar correctamente los datos y continuar con las investigaciones, es necesario implementar métodos eficientes que permitan ordenar grandes volúmenes de información de manera rápida.

En este taller, su tarea como ayudante del laboratorio será implementar distintos algoritmos de ordenamiento que permitan organizar los datos de los Pokémon según distintos criterios. A través de estos ejercicios, se busca analizar el comportamiento de distintos algoritmos de sorting y comprender cuándo es conveniente utilizar cada uno de ellos dependiendo del tamaño y características de los datos.

¡Orden en la Pokedex!

En el laboratorio del Profesor Oak se reciben constantemente nuevos registros de Pokemon provenientes de distintas regiones. Cada investigador envía información sobre los Pokemon que ha observado, incluyendo atributos como su ID en la pokedex, su nivel y su velocidad.

Debido a la gran cantidad de datos recibidos, la información llega completamente desordenada. Para poder analizar correctamente estos registros y compararlos entre distintas investigaciones, el laboratorio necesita ordenar los datos de forma eficiente.



Problema

El Profesor Oak ha reunido varios registros de Pokemon en arreglos de datos. Dependiendo del tamaño del conjunto de datos, distintos algoritmos de ordenamiento pueden ser más o menos eficientes.

Utilizando tus conocimientos de programación, deberás implementar distintos algoritmos de sorting para ordenar los registros del laboratorio y facilitar su análisis.

Entidades

Pokemon: Cada Pokémon cuenta con un `ID` único, un valor de `nivel` y un valor de `velocidad`. Todos los atributos son valores numéricos.

Flujo

Primero recibirás una línea con el entero E, que representa la cantidad de eventos a procesar.

Luego se recibirán E eventos, los cuales corresponden a distintas operaciones que el laboratorio del Profesor Oak necesita realizar sobre los registros de Pokemon.

Cada evento tendrá el nombre que indica el tipo de operación a ejecutar. Dependiendo del tipo de evento, se entregará la información necesaria para realizar la operación correspondiente.

Los eventos deberán ser procesados en el mismo orden en que aparecen en el input.

Para los eventos que requieran ordenar información, recibirás un entero N que representa la cantidad de Pokémon a procesar, seguido de N líneas con la información correspondientes al Pokemon.

Dependiendo del tipo de evento, deberás aplicar el algoritmo de ordenamiento correspondiente y mostrar el resultado solicitado.

Parte Formativa - Eventos

`SMALL_SORT {N} {atr} {orden}`

En este evento deberás ordenar los N Pokémon que se te entregaran según el atributo indicado en `atr`, que puede ser `NIVEL` o `VELOCIDAD`. Junto a el parámetro `orden`, que puede ser `ASC` o `DESC`, indicando si el ordenamiento debe realizarse de forma ascendente o descendente. Todo esto siguiendo con N lineas de los Pokemon a registrar con sus respectivos `ID`, `NIVEL`, `VELOCIDAD`.

En caso de empate en ambos atributos seleccionados, deberás desempatar con respecto al orden entregado utilizando el `ID` del Pokémon, el cual es único.

Este evento corresponde a conjuntos de datos pequeños, por lo que se espera que utilices un algoritmo de ordenamiento con complejidad aproximada $O(n^2)$.

El resultado esperado debe mostrar el siguiente output:

output

```
{N} Pokemones ordenados por {atr} en forma {orden}:
    Pokemon ID {Pok_id}: {atr} {Pok_atr}
    Pokemon ID {Pok_id}: {atr} {Pok_atr}
    .
    .
    .
    Pokemon ID {Pok_id}: {atr} {Pok_atr}
```

Donde atr podria ser tanto VELOCIDAD como NIVEL segun el caso.

`BIG_SORT {N} {atr_1} {atr_2} {orden}`

En este evento deberás ordenar los N Pokémon que se te entregaran según el primer atributo indicado en `atributo_1`, que puede ser `NIVEL` o `VELOCIDAD`. En caso de empate con el primer atributo se debera hacer la comparacion con el segun atributo `atributo_2`, que tambien puede ser `NIVEL` o `VELOCIDAD`. Todo junto a el parámetro `orden`, que puede ser `ASC` o `DESC`, indicando si el ordenamiento debe realizarse de forma ascendente o descendente. FInalmente siguiendo con N lineas de los Pokemon a registrar con sus respectivos `ID`, `NIVEL`, `VELOCIDAD`.

En caso de empate en ambos atributos seleccionados, deberás desempatar con respecto al orden entregado utilizando el `ID` del Pokémon, el cual es único.

Este evento corresponde a conjuntos de datos grandes, por lo que se espera que utilices un algoritmo de ordenamiento con complejidad aproximada $O(n * logn)$ para que sea eficiente.

El resultado esperado debe mostrar el siguiente output:

output

```
{N} Pokemones ordenados por {atr\_1} {atr\_2} en orden {orden}:
    Pokemon ID {pok_id}: {atr_1} {pok_atr_1} {atr_2} {pok_atr_2}
    Pokemon ID {pok_id}: {atr_1} {pok_atr_1} {atr_2} {pok_atr_2}
    .
    .
    .
    Pokemon ID {pok_id}: {atr_1} {pok_atr_1} {atr_2} {pok_atr_2}
```

Se puede suponer que `atr_1` y `atr_2` siempre seran distintos.

Parte Evaluada - Grupo 2

`DISTINTO {N}`

En el laboratorio del Profesor Oak, los investigadores están interesados en analizar la diversidad de velocidades de los Pokémon observados en distintas regiones. Para esto, necesitan saber cuántos valores distintos de `VELOCIDAD` aparecen en los registros.

Se te entregarán N líneas, cada una representando un Pokémon mediante sus atributos: `ID NIVEL VELOCIDAD`

Para este ejercicio, solo deberás considerar el atributo `VELOCIDAD`. Es decir, no importa el `ID` ni el `NIVEL` al momento de resolver el problema.

Tu tarea será contar cuántas velocidades distintas aparecen en el arreglo.

Ejemplo:

Si las velocidades recibidas fueran:

`1 2 8 5 3 2 6 5 4 8 7 3 6 5 50 32`

Entonces la cantidad de velocidades distintas serian:

`10`

El resultado esperado debe mostrar el siguiente output:

output:

```
Cantidad de velocidades distintas entre los {N} Pokemones: {X}
```

`INVERSION {N} {atr_1} {atr_2} {orden}`

Una inversión se define como un par de posiciones (i, j) tales que $i < j$ y el Pokémon en la posición i debería quedar después que el Pokémon en la posición j según el criterio de ordenamiento definido. Esto ayuda a medir que tan desordenado es un arreglo entregado.

En este evento deberás analizar qué tan desordenado se encuentra un conjunto de Pokémon recibido en el laboratorio y ordenarlo según el criterio indicado. Se te entregarán N líneas, cada una representando un Pokémon mediante sus atributos ID, NIVEL y VELOCIDAD, respectivamente.

Los Pokémon deberán ordenarse según el siguiente criterio:

- primero por `atr_1`,
- en caso de empate, por `atr_2`,
- y si el empate persiste, por `ID`, el cual es único.

El parámetro `orden` indica si el ordenamiento debe realizarse de forma ascendente (`ASC`) o descendente (`DESC`).

Además, deberás calcular la cantidad de **inversiones** que existen en el arreglo.

Importante: Este evento corresponde a un conjunto de datos grandes, por lo que se espera una complejidad aproximada de $O(n \log n)$. Por lo mismo, deberan modificar la implementacion de MergeSort **(funciones entregadas en su codigo base)** para que, ademas de ordenar los Pokemon, permita contabilizar las inversiones detectadas durante el proceso.

Pista:

- Aprovecha que ambas mitades ya están ordenadas al momento de mezclarlas.

Ejemplo:

$$[12, 10, 5, 3, 8]$$

$$[12, 10, 5] \quad [3, 8]$$

$$[12, 10, 5] \quad [8, 3] \quad inversion_count = 1$$

$$[12, 10, 8, 5, 3] \quad inversion_count = 2$$

Se espera el siguiente output para este evento:

`output:`

```
{N} Pokemones ordenados con {X} inversiones
por {atr_1} {atr_2} en orden {orden}:
    Pokemon ID {pok_id}: {atr_1} {pok_atr_1} {atr_2} {pok_atr_2}
    Pokemon ID {pok_id}: {atr_1} {pok_atr_1} {atr_2} {pok_atr_2}
    .
    .
    .
    Pokemon ID {pok_id}: {atr_1} {pok_atr_1} {atr_2} {pok_atr_2}
```

Ejecución

Tu programa se debe compilar con el comando `make` y debe generar un ejecutable de nombre `pokemones` que se ejecuta con el siguiente comando:

```
./pokemones input output
```

donde `input` será un archivo con los eventos a simular y `output` el archivo donde se guardarán los resultados.

En caso de querer correr el programa para ver los leaks de memoria utilizando valgrind deberás utilizar el siguiente comando:

```
valgrind ./pokemones input output
```

Tu tarea será ejecutada con archivos de creciente dificultad, asignando puntaje a cada una de estas ejecuciones que tenga un output igual al esperado.

Input

El archivo de input comenzará con el número `E` que corresponde al número de eventos a recibir, y las `E` líneas correspondientes a cada uno. Para cualquiera de los dos eventos, se entregara tambien un numero `N`, que indica la cantidad de pokemones a registrar a continuacion .

Como ejemplo tenemos el siguiente input:

input

```
2 # Cantidad de eventos
INVERSION {5} {VELOCIDAD} {NIVEL} {DESC}
1 11 12 # primer pokemon a registrar de los 5
5 12 10
4 1 5
6 56 3
10 5 8 # ultimo pokemon a registrar de los 5
DISTINTO 4
2 6 4 # primer pokemon a registrar de los 4
9 5 2
1 2 2
5 2 2 # ultimo pokemon a registrar de los 4
```

Output

output

```
5 Pokemones ordenados con 2 inversiones
por VELOCIDAD NIVEL en orden DESC:
    Pokemon ID 1: VELOCIDAD 12 NIVEL 11
    Pokemon ID 5: VELOCIDAD 10 NIVEL 12
    Pokemon ID 10: VELOCIDAD 8 NIVEL 5
    Pokemon ID 4: VELOCIDAD 5 NIVEL 1
    Pokemon ID 6: VELOCIDAD 3 NIVEL 56
Cantidad de velocidades distintas entre los 4 Pokemones: 2
```

Evaluación

La nota se calcula usando testcases especificos de evaluacion segun: 1 punto por output correcto y 0 puntos por output incorrecto.

Para cada test de evaluación, tu programa deberá entregar el output correcto en **menos de 2 segundos** y utilizar menos de 1 GB de RAM[1], de lo contrario recibirás 0 puntos en ese test. Por último, está **estrictamente prohibido** el uso de la función `realloc`, cuyo uso implicará un descuento de 5 décimas.

Integridad académica e IA

Cualquier acto deshonesto o fraude academico esta estrictamente prohibido. Se prohibe el uso de herramientas de inteligencia artificial y el uso o manipulacion de otros dispositivos electronicos como laptops, tablet, telefonos durante la evaluacion.

Proceso de Entrega

Conexion a internet

Una vez iniciado sesion en el escritorio, seleccionar el icono de redes arriba a la derecha. "*availablenetwork*" → "*eduroam*". Para conectarse a eduroam, deben tener la siguiente configuracion:

- Tener seleccionado la opcion "No CA certificate is required".
- En la opcion de "*Inner authentication*" seleccionar "*PAP*"
- Luego debe ingresar tus credenciales usuario uc. **IMPORTANTE:** en el usuario debes incluir el @, no solo el nombre.
- Ejemplos correctos: "*nombre@uc.cl*";"*nombre@estudiante.uc.cl*", Incorrecto: "nombre"sin el @.

Descargar taller

Para descargar el codigo base que con el que trabajaran, deben abrir una terminal, estar conectados a internet y ejecutar `descargar-taller nombre_archivo.zip`

Donde el *nombre_archivo* debe ser reemplazado con el nombre que se avisara al inicio de cada taller.

Editor de código

El editor de codigo de la imagen se llama "$lite - xl$". Para abrir su trabajo tienen varias alternativas, las recomendables son:

- Abrir el editor de codigo, buscar la seccion de "$file$" y seleccionar "*Open*" y buscar la carpeta donde quieres trabajar.
- Ir al explorador de archivos, buscar la carpeta donde quieres trabajar, hacer click derecho en la carpeta y seleccionar "*Openwith...*" y seleccionar el editor.
- **IMPORTANTE:** El editor no tiene autoguardado, por lo que cada vez que hagan cambios deben usar "$CTRL + S$" para guardar. Si ustedes no guardan sus archivos e intentan compilar, el proceso de compilado no guarda automáticamente, por lo que su código se compilara con la ultima versión que hayan guardado, **DEBEN GUARDAR ANTES DE COMPILAR**.

[1] Puedes revisarlo con el comando `htop` u ocupando `valgrind --tool=massif`

Espacio de trabajo

Al abrir el explorador de archivos o una terminal, veran una carpeta llamada código. Es **FUNDAMENTAL** que todo el trabajo que hagan lo tengan dentro de esa carpeta. Eso les asegura que en caso de que la imagen falle y tengan que reiniciar el computador, todo lo que está guardado en esa carpeta se mantendrá.

Uso del entorno de trabajo

En su carpeta de trabajo tendran al menos los siguientes elementos:

- src: carpeta con su codigo
- makefile: archivo para compilar
- tests: carpetas de tests publicas
- run.sh: script para ejecutar todos los tests

Para compilar, en una terminal usen:

```
make
```

Para usar el script que ejecuta todos los tests, usan:

```
./run.sh
```

El run.sh genera una carpeta llamada outputs que guarda para cada test, el output de su programa. **Si desean ejecutar un unico test** ocupan

```
./ejecutable ruta_al_input_del_test.txt archivo_output.txt
```

Donde ejecutable es el nombre del archivo que se genera al compilar, puedes revisarlo en las carpetas dentro de src, ejemplo de ejecutables: "pokemones", "magnemites". Ejemplo de ruta al archivo de input: tests/F1/pokemones/easy-0/input.txt y output.txt es un archivo txt que crean y es donde ira su salida.

En caso de querer debugear algun error de un test en especifico usar **valgrind**

```
valgrind ./ejecutable ruta_a_input.txt output.txt
```

Entrega Taller

Para entregar, tambien es importante que esten conectados a internet, entrar en una terminal, posicionarse en la ruta donde esta la carpeta de su entrega (no dentro de la carpeta misma). Finalmente, ejecutar

```
entregar-taller nombre_carpeta TX nAlumno1_nAlumno2_...
```

Donde: "*nombre_carpeta*" es el nombre de la carpeta que quieren entregar, por ejemplo: "*T*2*_evaluado_G*2". "*TX*" donde X correspondeal taller que se esta entregando: por ejemplo: "T2". Y *nAlumno*1*_nAlumno*2*_...* que son los numeros de alumnos de los integrantes del grupo que trabajaron separados por "_" (Si estan realizando este taller de manera solitaria, solamente escriben su numero de alumno).
Ejemplo Correcto:

12345678_23456789 20387645 20387645_12345678_23456789

Ejemplo Incorrecto:

1234567823456789 *Nombre*1_12345678 *Nombre*1*_Nombre*2*_Nombre*3